

Reconsider R0 of P3774 (Rename `std::nontype`) for C++26

Document #: P3843R0
Date: 2025-09-17
Project: Programming Language C++
Audience: LEWG
Reply-to: Jonathan Müller
<foonathan@jonathanmueller.dev>

Contents

- [1 Abstract](#)
- [2 Background](#)
- [3 Proposal](#)
- [4 Wording](#)
- [5 References](#)

§ 1 Abstract

Don't rename `std::nontype_t` to `std::constant_arg_t` as proposed by [\[P3774R1\]](#). Instead, rename it to something like `std::function_wrapper` and make it callable, as originally proposed by [\[P3774R0\]](#).

§ 2 Background

[\[P0792R14\]](#), adopted for C++26, adds `std::function_ref`, a non-owning reference wrapper for callable objects. To make it usable with member function pointers, it has an overload that takes a member function pointer using a `std::nontype_t` parameter. That way, we can encode the member function into the type system, making it possible to reference it without needing to keep the member function pointer alive.

Meanwhile, [\[P2841R1\]](#) renamed the core language term “non-type template parameter” to “constant template parameter”, so the naming of `std::nontype_t` should be reconsidered.

It also seemed that the `std::nontype_t` utility has become redundant with the adoption of [\[P2781R9\]](#)'s `std::constant_wrapper`. Like `std::nontype_t`, it allows encoding of a value into the type system, but it has features that make it more ergonomic to use. For

example, it overloads all operators of the underlying type, automatically re-wrapping the result in a `std::constant_wrapper` again.

However, as [P3792R0] points out, `std::constant_wrapper` is not a good replacement for `std::nontype_t` due to the fact that `std::constant_wrapper` is a callable itself. It overloads `operator()` with the same semantics as other operators: require that the operands are `std::constant_wrapper`'s themselves, unwrap them, forward to the underlying values, and re-wrap the result. This means that, as of the C++26 working draft, given `int fn(int)`, we have the following options to construct a `std::function_ref`:

- `function_ref(fn)` creates a function ref to `fn` with signature `int(int)`
- `function_ref(std::nontype<fn>)` creates a function ref to `fn` with signature `int(int)`
- `function_ref(std::cw<fn>)` creates a function ref to `std::cw<fn>` with signature `std::cw<int>(std::cw<int>)` (and also stores a dangling pointer to the `std::cw` object, but that's not the point here)

Therefore, using `std::constant_wrapper` instead of `std::nontype_t` would change the semantics of the function `std::constant_wrapper` naturally represents: Instead of a function that takes and returns `std::constant_wrapper`'s, it's a function that takes and returns the underlying type. In particular, it would be inconsistent with the behavior in e.g. `std::function`.

As such, [P3774R0] proposed to rename `std::nontype_t` to `std::fn_t` in C++26 and give it a call operator in C++29. This approach did not have consensus, so following LEWG guidance [P3774R1] pivoted to rename it to `std::constant_arg_t` instead without a call operator.

§ 3 Proposal

If [P3774R1] is adopted, we would have both `std::constant_wrapper` and `std::constant_arg_t` with the latter being used in `std::function_ref`. Both types have a very similar name and purpose: a library solution to the lack of “constexpr function parameters”. While `std::constant_wrapper` provides operator overloads making it useful to use with generic functions that want to work on both compile-time and runtime values, `std::constant_arg_t` does not overload anything, in particular not `operator()`, so it can be used with `std::function_ref`.

That would be embarrassing, frankly.

Do we seriously expect to teach people when to use one over the other?

Furthermore, it fragments the ecosystem: If some people choose to accept `std::constant_wrapper` for their compile-time parameters, and some others choose to use `std::constant_arg_t`, the lack of conversion between the two means that users will

have to potentially convert between the two representations of compile-time constants. Instead of one vocabulary types, we have two.

The idea behind [P3774R0] is much nicer: Renaming `std::nontype_t` to something like `std::fn_t` and making it callable gives it an identity distinct from `std::constant_wrapper`: `std::fn_t` is a compile-time known function, while `std::constant_wrapper` is a generic compile-time value. Furthermore, `std::fn_t` with a call operator is broadly useful beyond `std::function_ref`:

- It can be used to specify a function as the comparison or hasher of a `std::[unordered_]map`. Currently, you have to use a lambda to avoid the overhead of storing the pointer. With `std::fn_t` you just specify the function as template parameter.
- When passing a function to generic algorithms, unless the entire algorithm is inlined, we suffer an indirect call. If we pass a `std::fn_t` instead, the call target is known at compile time, enabling further information.

We therefore should reconsider the adoption of [P3774R0] (not R1!). There was some resistance to naming it `std::fn_t` without also providing the call operator, so let's just provide both in C++26. Let's also rename it to `std::function_wrapper` as that name had the most consensus during the [LEWG telecon](#).

§ 4 Wording

Adapted from [P3774R0], relative to [N5008].

In 17.3.2 [version.syn], update the feature-test macros:

```
-#define __cpp_lib_function_ref 202306L // also in <functional>
+#define __cpp_lib_function_ref 20XXXXL // also in <functional>
+#define __cpp_lib_fn 20XXXXL // also in <functional>
```

In 22.2.1 [utility.syn], delete the declarations of `std::nontype` and `std::nontype_t`:

```
-// nontype argument tag
-template<auto V>
-struct nontype_t {
-    explicit nontype_t() = default;
-};
-template<auto V> constexpr nontype_t<V> nontype{};
```

In 22.10.2 [functional.syn], change the synopsis as follows:

```
namespace std {
    [...]

    // [func.identity], identity
    struct identity;
        freestanding
}
```

```

+ // [func.wrapper], constant function wrapper
+ template<auto f>
+ struct function_wrapper;
+ template<auto f> constexpr function_wrapper<f> fw;

// [func.not.fn], function template not_fn
template<class F> constexpr unspecified not_fn(F&& f);           //
    freestanding
template<auto f> constexpr unspecified not_fn() noexcept;     //
    freestanding

[...]
}

```

Between 22.10.12 [func.identity] and 22.10.13 [func.not.fn], insert a new subclause:

§ Constant function wrapper [func.wrapper]

```

template<auto f>
struct function_wrapper {
    explicit function_wrapper() = default;

    see below
};

```

- 1 Let fw be an object of type FW that is a (possibly const) specialization of function_wrapper, and let cf be a template parameter object (13.2 [temp.param]) corresponding to the constant template argument of FW. Then:
 - FW is a trivially copyable type, such that FW models semiregular and is_empty_v<FW> is true;
 - fw is a simple call wrapper (22.10.4 [func.require]) with no state entities and with the call pattern invoke(cf, call_args...), where call_args is an argument pack used in a function call expression (7.6.1.3 [expr.call]), except that any parameter of the function selected by overload resolution may be initialized from the corresponding element of call_args if that element is a prvalue;
 - for any type R and pack of types Args, both fw and std::move(fw) are convertible to:
 - R(*) (Args...) if is_invocable_r_v<R, decltype(cf), Args...> is true;
 - R(*) (Args...) noexcept if is_nothrow_invocable_r_v<R, decltype(cf), Args...> is true.

[Example:

```

bool is_even(long);
bool(*ptr)(int) = std::fw<&is_even>;

```

— end example]

In 22.10.17.6 [\[func.wrap.ref\]](#), replace every occurrence of `nontype_t` with `function_wrapper`.

§ 5 References

[N5008] Thomas Köppe. 2025-03-15. Working Draft, Programming Languages — C++.

<https://wg21.link/n5008>

[P0792R14] Vittorio Romeo, Zhihao Yuan, Jarrad Waterloo. 2023-02-09. `function_ref`: a non-owning reference to a Callable.

<https://wg21.link/p0792r14>

[P2781R9] Zach Laine, Matthias Kretz, Hana Dusíková. 2025-06-17. `std::constexpr_wrapper`.

<https://wg21.link/p2781r9>

[P2841R1] Corentin Jabot, Gašper Ažman. 2023-10-14. Concept Template Parameters.

<https://wg21.link/p2841r1>

[P3774R0] Jan Schultke, Bronek Kozicki, Tomasz Kamiński. 2025-07-15. Rename `std::nontype`, and make it broadly useful.

<https://wg21.link/p3774r0>

[P3774R1] Jan Schultke, Bronek Kozicki, Tomasz Kamiński. 2025-08-14. Rename `std::nontype`, and make it broadly useful.

<https://wg21.link/p3774r1>

[P3792R0] Bronek Kozicki. 2025-07-13. Why ``constant_wrapper`` is not a usable replacement for ``nontype``.

<https://wg21.link/p3792r0>